

Metaheuristic Algorithms

Simulated Annealing and Tabu Search

Mateusz Smuga

12.09.2026

Methods

Data

The task was performed using data from the [University of Waterloo Department of Mathematics website](#). All used files are located in the `assets/` folder. The following datasets were utilized in this work:

Dataset	Number of Cities	File Name
Western Sahara	29	wi29.tsp
Djibouti	38	dj38.tsp
Qatar	194	qa194.tsp
Uruguay	734	uy734.tsp
Zimbabwe	929	zi929.tsp
Oman	1979	mu1979.tsp
Canada	4663	ca4663.tsp
Tanzania	6117	tz6117.tsp
Egypt	7146	eg7146.tsp
Ireland	8246	ei8246.tsp

Neighborhood

For both methods, a 2-opt neighborhood was chosen, where two intersecting edges are cut and untangled in hopes of achieving a shorter total route. It was implemented in the `math_util.rs` file along with the necessary helper functions.

Simulated Annealing Method

The method utilizes the concepts of temperature, epoch, and neighborhood. All code snippets discussed in this section are located in the `annealing.rs` and `simulate_annealing.rs` files.

Temperature

Temperature is a function of “time” that allows us to make suboptimal moves so as not to get trapped in a local maximum. Over time, it decreases, and “bad” moves occur less and less frequently. In this implementation, the initial temperature is chosen based on the data size and a constant selected by the user.

```
impl SimulationConstants {  
    pub fn from(p: &SimulationParameters) -> Self {  
        let n = p.size as f64;  
  
        let base_temperature = p.base_temperature_k * n.ln();  
    }  
}
```

`base_temperature_k` – parameter chosen before running the algorithm

n – size of the loaded data

The temperature itself is appropriately updated using the `update_temperature` function:

```
pub fn update_temperature(  
    temperature: &mut f64,  
    sim_consts: &SimulationConstants,  
) {  
    *temperature *= sim_consts.temp_constant.exp();  
}
```

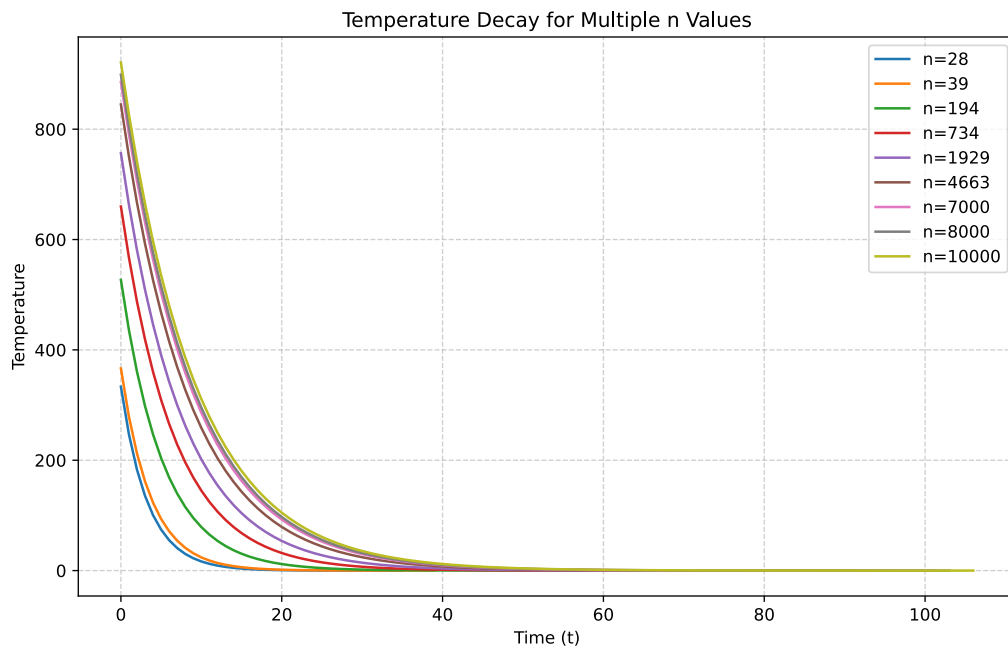
Note:

```
let tau = p.tau_k * base_temperature;  
let temp_constant = (-1.0)/n.ln();  
let min_delta = p.min_delta;
```

This recursive relationship can be described by the formula:

$$T(t, n) = \begin{cases} k \cdot \ln(n) & : t = 0 \\ T(t-1, n) \cdot e^{-\frac{1}{\ln(n)}} & : t > 0 \end{cases}$$

A chart comparing the behavior of the temperature function for different data sizes and standard constants has also been prepared:



Epoch

An epoch refers to all iterations sharing the same temperature. Here, the number of iterations within a given epoch can be adjusted using the following parameters:

```
pub epoch_min: u64,  
pub epoch_max: u64,
```

```
pub epoch_range: u64,
```

They are selected in the following manner:

```
let epoch_min = (p.epoch_k * n) as u64;
let epoch_max = (5.0 * p.epoch_k * n) as u64;
let epoch_range = epoch_max - epoch_min;
```

epoch_k – parameter chosen by the user

Stopping Condition

The stopping condition is quite trivial compared to the choice of the temperature function; namely, it suffices for the temperature to drop sufficiently low or for no improvement to be observed over the last 50 epochs:

```
pub fn should_stop(
    temp: f64,
    avg_delta: f64,
    no_improve_epochs: u64,
    p: &SimulationConstants,
) -> bool {
    temp < 0.001 || no_improve_epochs >= 50
}
```

Algorithm Execution

The execution of the algorithm can be described by the following steps:

1. Draw an initial solution X at random.
2. Choose a random solution X' located in the neighborhood of X .
3. If X' is better, accept it. Otherwise, calculate the probability of accepting X' using the formula:

$$\exp\left(\frac{f(X) - f(X')}{T}\right)$$

and with exactly that probability, set $X := X'$.

4. If the epoch has not ended, return to step two.
5. Decrease the temperature.
6. If the stopping condition has not been met, return to step two.

In code, this looks as follows:

```
pub fn run_simulation(cities: &[City], output_path: &str) -> f64 {
    let n = cities.len();
    if n == 0 { return 0.0; }

    let params = SimulationParameters::new(n as u64);
    let constants = SimulationConstants::from(&params);
    let dist_matrix = create_distance_matrix(cities);

    let mut current_path = generate_random_path(n);
    let mut current_distance = calculate_total_distance(&current_path, &dist_matrix);

    let mut best_path = current_path.clone();
    let mut best_distance = current_distance;
```

```

    let mut temperature = constants.base_temperature;
    let mut no_improve_epochs = 0;
    let mut total_iterations = 0;

    println!("Starting SA for {} cities. Initial distance: {:.2}", n,
current_distance);

    while !should_stop(temperature, current_distance, no_improve_epochs, &constants)
    {
        let mut improved_in_epoch = false;
        let current_epoch_len = epoch_length(temperature, &constants);

        for _ in 0..current_epoch_len {
            let (i, j) = get_random_indices(n);

            let delta = calculate_2opt_delta(&current_path, i, j, &dist_matrix);

            if accept(delta, temperature, &constants) {
                apply_2opt(&mut current_path, i, j);
                current_distance += delta;

                if current_distance < best_distance {
                    best_distance = current_distance;
                    best_path = current_path.clone();
                    improved_in_epoch = true;
                }
            }
            total_iterations += 1;
        }

        if improved_in_epoch {
            no_improve_epochs = 0;
        } else {
            no_improve_epochs += 1;
        }

        update_temperature(&mut temperature, &constants);
    }

    println!("Simulation finished. Best distance: {:.2} (Total iterations: {})",
best_distance, total_iterations);

    if let Err(e) = save_tour(&best_path, output_path) {
        eprintln!("Failed to save tour: {}", e);
    }

    best_distance
}

```

Parameter Tuning

The parameters used in the simulated annealing method were chosen empirically through manual testing so that the results converge to optimal solutions in the shortest possible time. The selected constants are:

```

impl SimulationParameters {
    pub const fn new(size: u64) -> Self {

```

```

Self {
  size,
  base_temperature_k: 100.0,
  epoch_k: 500.0,
  tau_k: 1.0,
  epsilon: 1e-7,
  min_delta: -1e-9,
}
}
}

```

Tabu Search Method

Algorithm

The algorithm extends the idea of Local Search by adding a tabu list and a smarter neighborhood search. The execution of the algorithm can be described as follows:

Randomly generate an initial solution Search its neighborhood and select the best solution that is not on the tabu list **Place it on the list and accept it as the current solution** Check the stopping condition

In code, this corresponds to the method from the `tabu_search.rs` file.

Stopping Condition

The stopping condition is the completion of a maximum number of iterations.

Neighbor Selection Method

A neighbor for analysis is chosen randomly by selecting two different vertices and attempting to perform an invert.

Parameter Tuning

Parameters chosen empirically through manual testing:

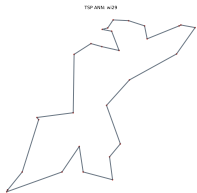
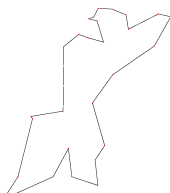
```

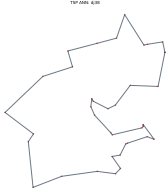
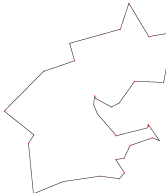
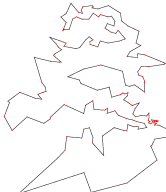
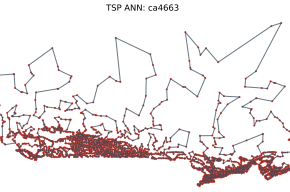
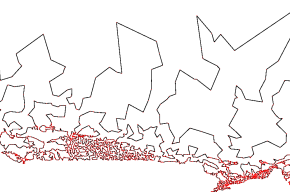
let tabu_params = TabuParameters {
  max_iterations: 10000,
  tabu_tenure: (n as f64 * 0.2) as usize,
  neighborhood_size: (n * 2).min(500),
};

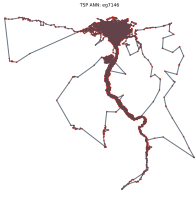
```

Obtained Results

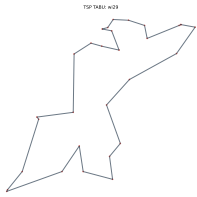
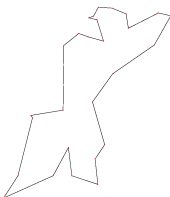
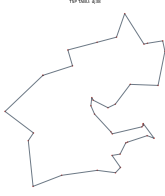
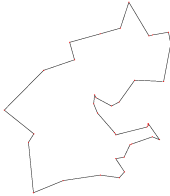
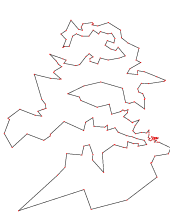
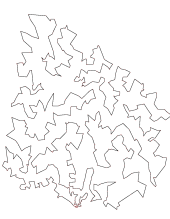
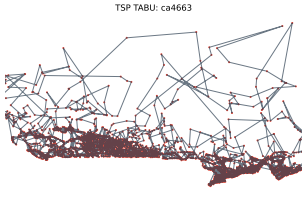
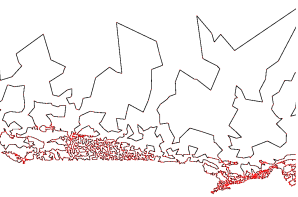
Simulated Annealing Algorithm Results

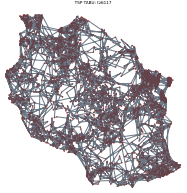
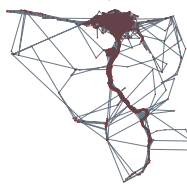
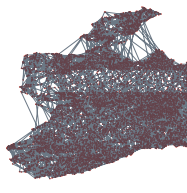
Instance (N)	Results (ANN)	Optimal Result	Length Found	Optimal Length
Western Sahara (wi29)			27601.17	27603

Djibouti (dj38)			6659.43	6656
Qatar (qa194)			9437.73	9352
Uruguay (uy734)			81324.27	79114
Zimbabwe (zi929)			97443.77	95345
Oman (mu1979)			89769.91	86891
Canada (ca4663)			1355651.52	1290319
Tanzania (tz6117)			417762.54	259045

Egypt (eg7146)			181049.5	501507
Ireland (ei8246)			219994.85	206171

Tabu Search Algorithm Results

Instance (N)	Results (TABU)	Optimal Result	Length Found	Optimal Length
Western Sahara (wi29)			27601.17	27603
Djibouti (dj38)			6659.43	6656
Qatar (qa194)			10039.7	9352
Uruguay (uy734)			93830.3	79114
Zimbabwe (zi929)			109839.34	95345
Oman (mu1979)			134567.19	86891
Canada (ca4663)			3980677.91	1290319

Tanzania (tz6117)			1841657.65	259045
Egypt (eg7146)			965296.17	501507
Ireland (ei8246)			1479252.09	206171

Length Comparison

